Tutorial Questions | Week 9

COSC2779 - Deep Learning

This tutorial reviews representation learning, foundation models, and how to adapt them (transfer learning, fine-tuning, and parameter-efficient fine-tuning). Please try the questions before you join the session. The question below has a similar structure to the case study questions that you can expect in the end semester test.

1. You have been hired by a company that builds text-triage tools for a network of community legal centres across Australia. Each centre receives a large volume of free-text client emails, and staff must read every email to decide which legal area it concerns (e.g. *tenancy*, *employment*, *family*, *consumer*, ... – eight areas in total) so it can be routed to the right team.

   You are given:

   - a very large collection of **unlabelled** historical emails (millions), and
   - a small **labelled** set of about **2,000** emails, each tagged with its legal area.

   Constraints: a modest GPU budget, more tasks are planned later (urgency detection, entity extraction), and the setting is high-stakes and fairness-sensitive. You decide to build the system around a pre-trained Transformer.

   (a) Your first instinct is to represent each email by averaging word2vec/GloVe vectors of its words. Why is a contextual representation from a pre-trained Transformer usually better?

   > **Solution:**
   >
   > - word2vec/GloVe give **one fixed vector per word**, regardless of context – so a word's meaning cannot adapt to the sentence (e.g. "bond" as a rental deposit vs a financial bond).
   > - A pre-trained Transformer produces **contextual** embeddings: the vector for a word is computed from the whole email, capturing meaning, order and context.
   > - Averaging static vectors also discards word order; the Transformer keeps it (self-attention + positional encodings).

   (b) You have millions of unlabelled emails but only 2,000 labels. How can the unlabelled data help? Name a suitable objective.

   > **Solution:** Use **self-supervised pre-training** on the unlabelled emails: the data supplies its own labels.
   >
   > - e.g. **masked-token prediction** (mask ~15% of tokens, predict them from context) or next-token prediction.
   > - This learns general-purpose (and domain-adapted) language representations with no human labels; we then adapt with the small labelled set.

   (c) What is a **foundation model**, and give two reasons to start from one rather than train a bespoke model from scratch here?

   > **Solution:** A model **pre-trained on broad data at scale** (usually self-supervised) that can be adapted to many downstream tasks.
   >
   > - You have very little labelled data – a pre-trained model already "knows" language, so adaptation needs far fewer labels.
   > - Amortised cost / generality: pre-train once, reuse for the routing task and the later tasks (urgency, entities). Training from scratch would need huge data and compute.

   (d) List the main strategies for adapting a pre-trained model, ordered from cheapest to most expensive, one line each.

Solution:

- **Zero-shot**: use the model as-is (e.g. prompting) – no training.
- **Feature extraction**: freeze the model, train a small head on its features.
- **PEFT (e.g. LoRA)**: train a small number of extra parameters, backbone frozen.
- **Full fine-tuning**: update all weights.

(e) With only ∼2,000 labels and a modest GPU, which strategy (or strategies) would you try, and why?

Solution: Feature extraction and/or **PEFT (LoRA)** are the natural choices.

- They train few parameters – cheap on a small GPU and **less prone to over-fitting** 2,000 examples.
- **Full fine-tuning** risks over-fitting the small set and costs the most; try it only if the lighter methods underperform.
- Also run **zero-shot** as a quick baseline. Always validate empirically.

(f) Explain the difference between **feature extraction** and **full fine-tuning**, with one pro and one con of each.

Solution:

- **Feature extraction**: freeze the backbone, use its outputs as fixed features, train only a small head. Pro: fast, few parameters, robust on small data. Con: limited – the representation can't adapt to the task.
- **Full fine-tuning**: update all weights. Pro: best accuracy with enough data. Con: expensive (a full model per task), can over-fit, and risks **catastrophic forgetting**.

(g) Later you will serve many tasks. Why is full fine-tuning costly to store/serve per task, and how does **LoRA** address this? Write the low-rank update.

Solution: Full fine-tuning stores a **full copy of every weight per task** (plus optimizer state during training) – billions of parameters each.

LoRA freezes the pre-trained weight W_0 and learns a **low-rank** update:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k).$$

Only A, B are trained (often $< 1\%$ of parameters). Adapters are tiny and swappable per task, and can be **merged** into W_0 at inference (no extra latency).

(h) In LoRA, what does the rank r control, and what is the trade-off in increasing it?

Solution: r sets the **capacity** of the update (how expressive ΔW can be) and the **number of trainable parameters** ($r(d+k)$ per layer).

- Too small: may **underfit** the task.
- Larger: more capacity and usually a little more accuracy, but more parameters/compute – with diminishing returns.

(i) When could **zero-shot** give useful predictions with no training at all, and give one limitation.

Solution: If you use a capable generative model, you can **prompt** it ("classify this email into one of these areas: . . . ") and read off the answer – no labelled training. (Vision–language models like CLIP do the analogous thing for images.)

Limitation: sensitive to prompt wording and usually below a properly tuned model; it also inherits all of the base model's blind spots.

(j) This is a high-stakes, fairness-sensitive setting. Name two risks of using a foundation model here and one mitigation.

Solution:

- **Inherited bias** from web-scale pre-training → skewed error rates across groups (e.g. by dialect, name, postcode) – an **allocative harm** in routing.
- **Distribution shift / shortcut learning** or **hallucination** (if generative): confident but wrong routing on emails unlike the training data.

Mitigation: evaluate with **disaggregated** metrics across groups (not just aggregate accuracy), curate/balance the adaptation data, and keep **human oversight** for a high-stakes decision.